AWROR TECHNICAL DOCUMENTATION

Prepared By:

Joel Muñiz Expósito

ABOUT

Colexio Vivas

CSS Multiplatform Application Development

Vigo

INDEX

1. System architecture
2. API
3. Database
4. Web application
5. Mobile application
6. Desktop application
7. Security
8. Deployment
9. User and maintenance guide
10. Future improvements

1. System Arquitecture

• General schema

The AWROR system is composed by:

- Desktop application
- Web application
- Mobile application
- API Rest
- Database

Comunication flow:

1. The desktop application sends HTTP requests to the API.
2. The API processes the request, executes SQL queries in MariaDB, and returns JSON.
3. The application interprets the response and updates the interface.

• Principal technologies

Desktop application: C# .NET Windows Forms
Web application: PHP
Mobile application: Android Studio (Java)
API Rest: Java
Database: MariaDB

2. API

- ## Endpoints list

Users

Method	Endpoint	Description
GET	/users/{id}	Get user data
POST	/users/register	Create user
POST	/users/login	Log in user
PUT	/users/{id}	Edit usuario
DELETE	/users/{id}	Delete user

Pets

Method	Endpoint	Description
GET	/pets	Get Pets
POST	/pets/user/{id}	Create pet
PUT	/pets/{id}	Edit pet
DELETE	/pets/{id}	Delete pet

Posts

Method	Endpoint	Description
GET	/posts	Get posts
GET	/posts/user/{id}	Get user's post
POST	/posts	Create post
PUT	/posts/{id}	Editar post
DELETE	/posts/{id}	Eliminar post

Vaccines

Method	Endpoint	Description
GET	/vaccines/pet/{id}	Get pet's vaccines
POST	/vaccines	Crear vaccine
PUT	/vaccines/{id}	Edit vaccine
DELETE	/vaccines/{id}	Delete vaccine

• HTTP METHODS

POST: CREATE.
GET: READ.
PUT: UPDATE.
DELETE: DELETE.

} CRUD

• AUTHENTICATION

THE API USES:

- LOG IN BY USER/PASSWORD
- SESSION MANAGED IN THE CLIENT APPLICATION

• REQUEST EXAMPLES

CREATE PET: /PETS/USER/{ID}

REQUEST JSON

```
{
   "user_id": 1,
   "name": "Chips",
   “type”: “Dog”,
   “image”: “chips.jpg”
}
```

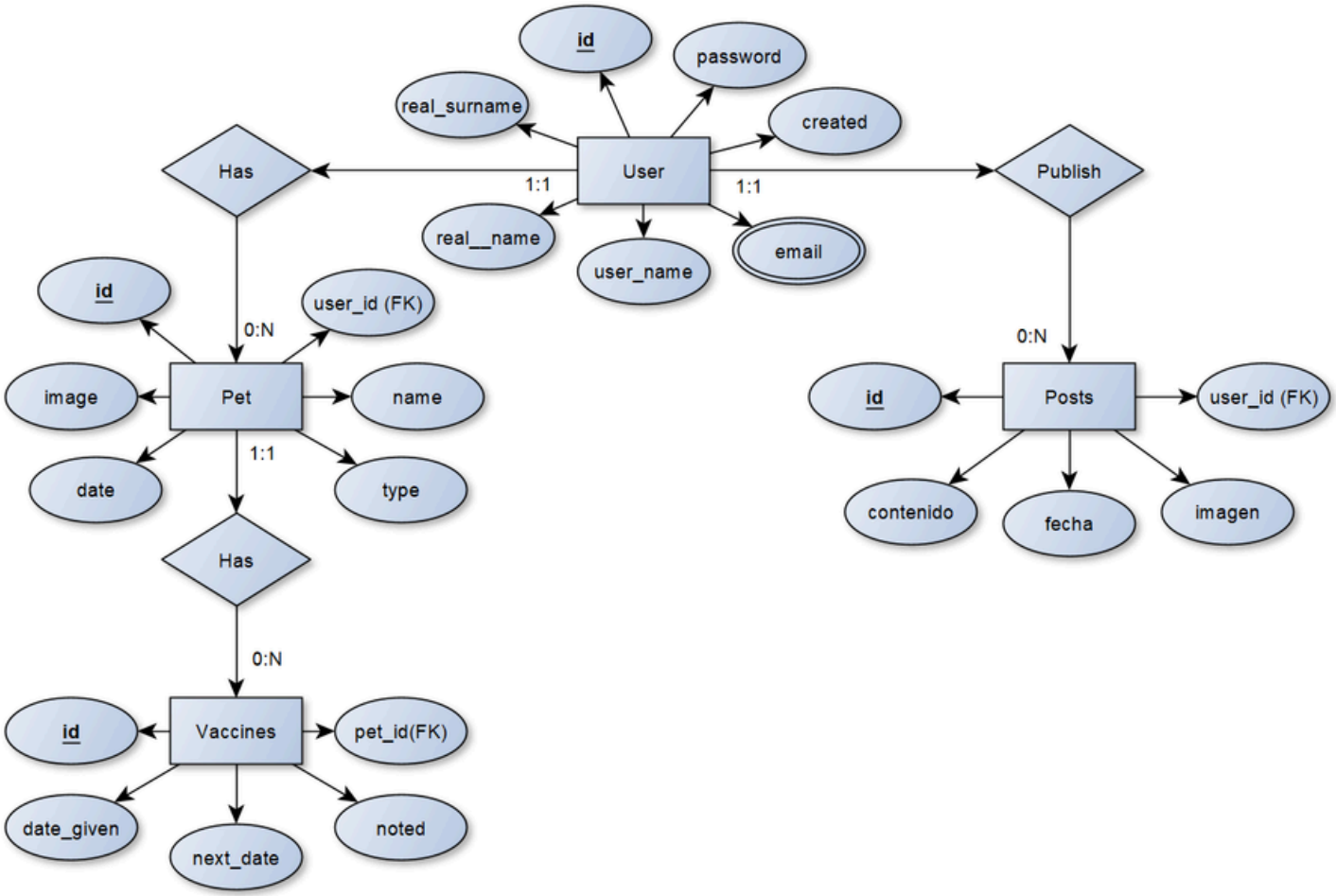
→

RESPONSE

“Pet created”

3. Database

- Entity-relantionship diagram



This is the entity-relationship diagram of the AWROR database, where you can see how the different tables are related to each other.

- ## Description of tables and fields

Users table

ID	INT
REAL_NAME	VARCHAR(255)
REAL_SURNAME	VARCHAR(255)
EMAIL	VARCHAR(255)
PASSWORD	VARCHAR(255)
USER_NAME	VARCHAR(255)
CREATED	DATETIME

The ID is the primary key and is also auto incremented

Pets table

Primary key auto incremented

User_id is a foreign key, references the field id in users table

id	int
user_id	int
name	varchar(100)
type	varchar(50)
image	blob
date	datetime

Posts table

Primary key auto incremented

User_id is a foreign key, references the field id in users table

id	int
user_id	int
name	text
image	blob

Primary key auto incremented

pet_id is a foreign key, references the field id in pets table

Vaccines table

id	int
pet_id	int
name	varchar(100)
date_given	datetime
next_date	datetime
notes	text

• Important rules

- A pet belongs to a single user.
- A vaccine belongs to a single pet.
- A post belongs to a single user.
- Images are stored as BLOB and sent as Base64.

• Migrations management

- Exportation/importation from phpMyAdmin.

4. Web Application

• Technologies employed

- PHP: principal language
- HTML: page structure
- Javascript: comprobations
- CURL: API communication
- Xampp: local execution
- MariaDB: database

• Project structure

```
/WEB
    ↳ INICIOSESION.PHP
    ↳ /VISTAS
        ↳ CREARCUENTA.PHP
        ↳ PERFIL.PHP
        ↳ MASCOTAS.PHP
        ↳ VACUNAS.PHP
        ↳ PRINCIPAL.PHP
```

• API connection

The app communicates with the API using CURL

```
$ch = curl_init("http://localhost:8080/apiawror/rest/users");
curl_setopt($ch, CURLOPT_CUSTOMREQUEST, "POST");
```

5. Mobile Application

• Technologies employed

- Android studio: Java for classes and XML for interfaces
- Apirest: backend
- Httpurlconnection: API communication

• Project structure

/app/src/main/java/com/example/awror_def

- crear_cuenta.java
- inicio_sesion.java
- pet.java (has adapter)
- post.java (has adapter)
- vaccine.java (has adapter)
- mainactivity.java (manage fragments)
- apirest.java

• API connection

The app communicates with the API using HttpURLConnection

```
URL url = new URL(BASE_URL + "users");
HttpURLConnection con = (HttpURLConnection) url.openConnection();
```

6. Desktop Application

• Technologies Employed

- C#: .NET Framework and Windows Forms
- HttpClient: API communication
- Newtonsoft.Json: JSON serialization

• Project Structure

/Awror

- CrearCuenta.cs
- IniciarSesion.cs
- Perfil.cs
- Post.cs (only class)
- Vacunas.cs (has dialog)
- Mascotas.cs (has dialog)
- Principal.cs (has dialog)
- Vacuna.cs (only class)
- Mascota.cs (only class)
- Sesion.cs (only class)

• API Connection

The app communicates with the API using HttpClient

```
HttpResponseMessage response = await client.PostAsync("users", content);
if (response.StatusCode == HttpStatusCode.Conflict)
{
    MessageBox.Show("El correo ya está registrado");
}
```

7. Security

• Current mechanisms

- User validation during login
- Permission control on endpoints (only the owner can edit/delete)
- Secure image conversion
- Parameterized SQL (prevents SQL Injection)

• Eliminated risks

- SQL Injection: Use of PreparedStatement

• Aplied good practices

- API / client separation
- Do not expose passwords
- Do not store images on the client's disk

8. Deployment

• How to execute each app

Desktop application:

1. Open the solution of the project in visual studio
2. Execute project

Mobile application:

1. Open the project in android studio
2. Execute the emulator

Web application:

1. Open the folder of the project in visual studio code
2. Execute project with php

• API deployment

1. Compile java project
2. Start tomcat server

• Environments

- Local environment

9. User and Maintenance Guide

• Local Execution

1. Start XAMPP (Apache + MySQL)
2. Start the Tomcat server in the folder that contains the API.
3. Start the mobile app in Android Studio or the web app in Visual Studio Code or the desktop app in Visual Studio.

• Add Features

1. Create API endpoint: Add the method in the respective handler.
2. Update the 3 Apps:
    a. Desktop: add API method
    b. Mobile: add method in ApiRest
    c. Web: add respective curl
3. Update documentation

• Common Issues

- Android Studio does not show bugs
- The app does not connect to the API

10. Future Improvements

• Planned Improvements

- Likes and comments in posts
- Add biography in the profile
- Add a profile photography

• New Features

- Shared pet among users
- Follow other users
- Direct message between users
- Adoption system
- Missing pets map

• Potential optimizations

- Image cache on android
- Tighter backend validation
- Color blindness option
- Dark mode